

Taylor & Francis
Taylor & Francis Group

Journal of Computational and Graphical Statistics

ISSN: 1061-8600 (Print) 1537-2715 (Online) Journal homepage: https://www.tandfonline.com/loi/ucgs20

A Gradient-Based Optimization Algorithm for LASSO

Jinseog Kim, Yuwon Kim & Yongdai Kim

To cite this article: Jinseog Kim, Yuwon Kim & Yongdai Kim (2008) A Gradient-Based Optimization Algorithm for LASSO, Journal of Computational and Graphical Statistics, 17:4, 994-1009, DOI: 10.1198/106186008X386210

To link to this article: https://doi.org/10.1198/106186008X386210

View supplementary material

Published online: 01 Jan 2012.

Submit your article to this journal

Article views: 317

View related articles

Citing articles: 2 View citing articles

Full Terms & Conditions of access and use can be found at
https://www.tandfonline.com/action/journalInformation?journalCode=ucgs20

Supplemental materials for this article are available online
http://pubs.amstat.org/doi/suppl/10.1198/106186008X386210/suppl_file/kim.ZIP

A Gradient-Based Optimization Algorithm for LASSO

Jinseog KIM, Yuwon KIM, and Yongdai KIM

LASSO is a useful method for achieving both shrinkage and variable selection simultaneously. The main idea of LASSO is to use the L_1 constraint in the regularization step which has been applied to various models such as wavelets, kernel machines, smoothing splines, and multiclass logistic models. We call such models with the L_1 constraint generalized LASSO models. In this article, we propose a new algorithm called the *gradient LASSO* algorithm for generalized LASSO. The gradient LASSO algorithm is computationally more stable than QP-based algorithms because it does not require matrix inversions, and thus it can be more easily applied to high-dimensional data. Simulation results show that the proposed algorithm is fast enough for practical purposes and provides reliable results. To illustrate its computing power with high-dimensional data, we analyze multiclass microarray data using the proposed algorithm.

Key Words: Gradient descent; LASSO; Multiclass logistic model; Variable selection.

1. INTRODUCTION

Tibshirani (1996) introduced an interesting method for shrinkage and variable selection in linear models, called the "Least Absolute Shrinkage and Selection Operator (LASSO)." The main idea of LASSO is to use the L_1 constraint in the regularization step. That is, the estimator is obtained by minimizing the empirical risk subject to the constraint that the L_1 norm of the regression coefficients is bounded by a given positive number. In addition to linear models, the idea of using the L_1 constraint has been applied to various models such as wavelets (Chen et al. 1999; Bakin 1999), kernel machines (Gunn and Kandola 2002; Roth 2004), smoothing splines (Zhang et al. 2004) and multiclass logistic regressions (Krishnapuram et al. 2004). We call such models with the L_1 constraint *generalized LASSO* models.

Jinseog Kim is an Assistant Professor, Department of Statistics and Information Science, Dongguk University, Gyeongju, 780-714, Korea (E-mail: *jinseog.kim@gmail.com*). Yuwon Kim is a Researcher, Datamining Lab., NHN Corp., Gyeonggi-Do, 463-844, Korea (E-mail: *yuwon.kim@nhncorp.com*). Yongdai Kim is an Associate Professor, Department of Statistics, Seoul National University, Seoul, 151-742, Korea (E-mail: *ydkim0903@gmail.com*).

© *2008 American Statistical Association, Institute of Mathematical Statistics, and Interface Foundation of North America*

Journal of Computational and Graphical Statistics, Volume 17, Number 4, Pages 994–1009
DOI: 10.1198/106186008X386210

One of the important issues in generalized LASSO is that the objective function is not differentiable due to the L_1 constraint. Hence special optimization techniques, such as those in Osborne et al. (2000a) and Efron et al. (2004) are required. However, such algorithms may fail to converge when the loss function is not the squared error loss. In fact, we have had many failures of convergence with the `lasso2` function in the R package, which uses a modified Osborne's algorithm developed by Lokhorst et al. (2006) for solving the L_1 constrained logistic regression problem. The main reason of the failure of convergence is that the algorithm involving the matrix inversions often fails to obtain the solution. Similar problems still exist in the recently developed optimization algorithms by Rosset and Zhu (2007) and Park and Hastie (2007).

Some attempts have been made to resolve these computational issues in generalized LASSO; see, for example, Grandvalet and Canu (1999) and Perkins et al. (2003). However, these algorithms may converge to suboptimal solutions and it is difficult to check whether the solution is optimal. In this article, we propose a new gradient-based optimization algorithm called the *gradient LASSO* algorithm. The proposed algorithm has the advantages that it never fails, and always converges to the optimal solution for general convex loss functions under regularity conditions.

The proposed algorithm is a refined version of the earlier gradient LASSO algorithm by Kim and Kim (2004). We add a deletion step, which greatly improves the convergence speed and provides more stable solutions. Ma and Huang (2005) applied the earlier version of the gradient LASSO algorithm to survival analysis.

This article is organized as follows. In Section 2, we explain why the existing algorithms can fail to converge. In Section 3, we propose the gradient LASSO algorithm. Section 4 illustrates the gradient LASSO algorithm on simulated and real datasets. We demonstrate how much the newly introduced deletion step improves the performance of the gradient LASSO algorithm, and we compare the gradient LASSO algorithm with the frequently used algorithm (`lasso2`; Lokhorst et al. 2006) in the logistic regression model by simulation. We analyze multiclass microarray data using the proposed algorithm. Concluding remarks follow in Section 5.

2. THE QP-BASED ALGORITHM: REVIEW

In this section, we review the Osborne's algorithm (Osborne et al. 2000b) for the squared error loss and its extension to general convex loss functions. The objective of this section is to explain why and when these algorithms fail to converge.

Let $(y_1, \boldsymbol{x}_1), \ldots, (y_n, \boldsymbol{x}_n)$ be n output/input pairs where $y_i \in \mathcal{Y}$ and $\boldsymbol{x}_i = (x_{i1}, \ldots, x_{ip})' \in \mathcal{X}$ where $\mathcal{X}$ is a subspace of R^p, the p-dimensional Euclidean space. Let $\boldsymbol{\beta} = (\beta_1, \ldots, \beta_p)'$ be the corresponding regression coefficients. Given loss function $\mathcal{L} : \mathcal{Y} \times \mathcal{X} \times R^p \to R$, the objective of generalized LASSO is to find $\boldsymbol{\beta}$ which minimizes the empirical risk

$$R(\boldsymbol{\beta}) = \sum_{i=1}^{n} \mathcal{L}(y_i, \boldsymbol{x}_i, \boldsymbol{\beta}) \tag{2.1}$$

subject to $||\boldsymbol{\beta}||_1 \leq \lambda$ where $\lambda > 0$ is a constrained parameter and $\|\boldsymbol{\beta}\|_1 = \sum_{k=1}^{p} |\beta_k|$ denotes the L_1 norm on R^p.

Osborne et al. (2000b) proposed the following algorithm when $\mathcal{L}(y, \boldsymbol{x}, \boldsymbol{\beta}) = (y - \boldsymbol{x}'\boldsymbol{\beta})^2$. For the current value $\boldsymbol{\beta}$, let $\sigma = \{j : \beta_j \neq 0\}$ be the index set of nonzero coefficients, and $|\sigma|$ denote the cardinality of σ. Let P represent the permutation matrix that collects the nonzero elements of $\boldsymbol{\beta}$ in the first $|\sigma|$ elements and write

$$\boldsymbol{\beta} = P^T \begin{pmatrix} \boldsymbol{\beta}_\sigma \\ \mathbf{0} \end{pmatrix}.$$

Let $\boldsymbol{\theta}_\sigma = \text{sign}(\boldsymbol{\beta}_\sigma)$ have entry 1 if the corresponding entry in $\boldsymbol{\beta}_\sigma$ is positive and -1 otherwise. The Osborne's algorithm consists of the following three steps: optimization, deletion, and optimality check and addition. In the optimization step, the algorithm solves

$$\begin{aligned} &\text{minimize}_{\boldsymbol{h}} \; R(\boldsymbol{\beta} + \boldsymbol{h}) \\ &\text{subject to } \boldsymbol{\theta}_\sigma^T(\boldsymbol{\beta}_\sigma + \boldsymbol{h}_\sigma) \leq \lambda \text{ and } \boldsymbol{h} = P^T \begin{pmatrix} \boldsymbol{h}_\sigma \\ \mathbf{0} \end{pmatrix}. \end{aligned} \tag{2.2}$$

It turns out that the solution of (2.2) is

$$\boldsymbol{h}_\sigma = (\mathbf{X}_\sigma' \mathbf{X}_\sigma)^{-1}\{\mathbf{X}_\sigma'(\boldsymbol{y} - \mathbf{X}_\sigma \boldsymbol{\beta}_\sigma) - \mu \boldsymbol{\theta}_\sigma\}, \tag{2.3}$$

where

$$\mu = \max\left\{0, \frac{\boldsymbol{\theta}_\sigma^T(\mathbf{X}_\sigma^T \mathbf{X}_\sigma)^{-1}\mathbf{X}_\sigma^T \boldsymbol{y} - \lambda}{\boldsymbol{\theta}_\sigma^T(\mathbf{X}_\sigma^T \mathbf{X}_\sigma)^{-1}\boldsymbol{\theta}_\sigma}\right\}.$$

Here $\boldsymbol{y} = (y_1, \ldots, y_n)'$ and $\mathbf{X}_\sigma$ is the matrix which consists of the first $|\sigma|$ columns of $P\mathbf{X}$ where $\mathbf{X}$ is the design matrix.

The deletion step checks whether $\boldsymbol{\beta}^\dagger = \boldsymbol{\beta} + \boldsymbol{h}$ is sign feasible, that is $\text{sign}(\boldsymbol{\beta}_\sigma^\dagger) = \boldsymbol{\theta}_\sigma$. If $\boldsymbol{\beta}^\dagger$ is sign feasible, the deletion step is terminated. Otherwise, we delete an element in σ, that most violates the sign feasibility until the remaining active coefficients satisfy the sign feasibility. Then, we check the optimality of the solution. If the current solution satisfies the optimality condition, the algorithm terminates. Otherwise, we update σ by adding an element that most violates the optimality condition, and return to the optimization step.

Lokhorst et al. (2006) modified the Osborne's algorithm for the generalized linear model including logistic regression model within the iterative reweighted least squares algorithm. They also built the R package called `lasso2`. Hereafter, we call it the `lasso2` algorithm.

The `lasso2` algorithm as well as Osborne's algorithm fail to converge when $\mathbf{X}_\sigma$ is singular, which frequently occurs when the dimension of inputs is high. See the simulation results in Section 4 for empirical evidence of this.

3. THE GRADIENT LASSO ALGORITHM

In this section, we will present a gradient descent algorithm to solve the optimization problem in generalized LASSO. Recall that the optimization problem of generalized

LASSO is to minimize the empirical risk $R(\boldsymbol{\beta})$ subject to $\|\boldsymbol{\beta}\|_1 \le \lambda$. We first introduce the coordinate-wise gradient descent (CGD) algorithm proposed by Kim and Kim (2004) and then present the gradient LASSO algorithm which speed up the CGD algorithm by adding a deletion step.

Remark. Kim and Kim (2004) used the name gradient LASSO for the CGD algorithm. The gradient LASSO algorithm in this article is an accelerated version of Kim and Kim (2004). This acceleration, however, has an important implication for sparsity; see Section 4.1 for details.

3.1 COORDINATE-WISE GRADIENT DESCENT ALGORITHM

Let $\boldsymbol{w} = \boldsymbol{\beta}/\lambda$ and $\mathcal{S} = \{\boldsymbol{w} : \|\boldsymbol{w}\|_1 \le 1\}$. Then the optimization problem of generalized LASSO is equivalent to minimizing $C(\boldsymbol{w}) = R(\lambda \boldsymbol{w})$ subject to $\boldsymbol{w} \in \mathcal{S}$, and so we are to find $\hat{\boldsymbol{w}}$ in $\mathcal{S}$ such that

$$\hat{\boldsymbol{w}} = \arg\min_{\boldsymbol{w}\in\mathcal{S}} C(\boldsymbol{w}).$$

The main idea of the CGD algorithm is to find $\hat{\boldsymbol{w}}$ sequentially as follows. Given $\boldsymbol{v} \in \mathcal{S}$ and $\alpha \in [0, 1]$, let

$$\boldsymbol{w}[\alpha, \boldsymbol{v}] = \boldsymbol{w} + \alpha(\boldsymbol{v} - \boldsymbol{w}).$$

Suppose that $\boldsymbol{w}$ is the current solution. The CGD algorithm first searches a direction vector $\boldsymbol{v}$ in $\mathcal{S}$ such that $C(\boldsymbol{w}[\alpha, \boldsymbol{v}])$ decreases most rapidly, and then it updates $\boldsymbol{w}$ to $\boldsymbol{w}[\alpha, \boldsymbol{v}]$. Note that $\boldsymbol{w}[\alpha, \boldsymbol{v}]$ is still in $\mathcal{S}$. The Taylor expansion implies

$$C(\boldsymbol{w}[\alpha, \boldsymbol{v}]) \approx C(\boldsymbol{w}) + \alpha\langle \nabla(\boldsymbol{w}), \boldsymbol{v} - \boldsymbol{w}\rangle,$$

where $\nabla(\boldsymbol{w}) = (\nabla(\boldsymbol{w})_1, \ldots, \nabla(\boldsymbol{w})_p)$ and $\nabla(\boldsymbol{w})_k = \partial C(\boldsymbol{w})/\partial w_k$. Here, $\langle \cdot, \cdot \rangle$ is the inner product. Moreover, it can be easily shown that

$$\min_{\boldsymbol{v}\in\mathcal{S}} \langle \nabla(\boldsymbol{w}), \boldsymbol{v}\rangle = \min_{k=1,\ldots,p} \min\{\nabla(\boldsymbol{w})_k, -\nabla(\boldsymbol{w})_k\}.$$

Hence, the desired direction is a vector in R^p such that $\hat{k}$th element is $-\text{sign}(\nabla(\boldsymbol{w})_{\hat{k}})$ and the other elements are zeros, where

$$\hat{k} = \arg\min_{k=1,\ldots,p} \min\{\nabla(\boldsymbol{w})_k, -\nabla(\boldsymbol{w})_k\}.$$

The CGD algorithm for generalized LASSO is described in Figure 1. For each iteration in the CGD algorithm, $\hat{\alpha}$ can be easily determined because the optimization problem at hand is an one-dimensional convex optimization.

Assume that C is convex and its gradient ∇ satisfies the Lipschitz condition with the Lipschitz constant L on $\mathcal{S}$: for any two vectors $\boldsymbol{w}$ and $\boldsymbol{v}$ in R^p,

$$\|\nabla(\boldsymbol{w}) - \nabla(\boldsymbol{v})\| \le L\|\boldsymbol{w} - \boldsymbol{v}\|, \tag{3.1}$$

where $\|\cdot\|$ is the Euclidean norm. Let $C^* = \inf_{\boldsymbol{w}\in\mathcal{S}} C(\boldsymbol{w})$ and $\Delta C(\boldsymbol{w}) = C(\boldsymbol{w}) - C^*$. Let $M = \max\{M_1, M_2\}$ where $M_1 = L \sup_{\boldsymbol{w},\boldsymbol{v}\in\mathcal{S}} \|\boldsymbol{w} - \boldsymbol{v}\|^2$ and $M_2 = \sup_{\boldsymbol{w}\in\mathcal{S}} C(\boldsymbol{w}) -$

1. **Initialize**: $\boldsymbol{w} = \mathbf{0}$ and $m = 0$.
2. **Do until converge**
   (a) $m = m + 1$.
   (b) Compute the gradient $\nabla(\boldsymbol{w})$.
   (c) Find the $(\hat{k}, \hat{\gamma})$ that minimizes $\gamma \nabla(\boldsymbol{w})_k$ for $k = 1, \ldots, p$ and $\gamma = \pm 1$.
   (d) Let $\boldsymbol{v}$ be the p dimensional vector such that the $\hat{k}$-th element is $\hat{\gamma}$ and the other column elements are zeros.
   (e) Find $\hat{\alpha} = \arg\min_{\alpha \in [0,1]} C(\boldsymbol{w}[\alpha, \boldsymbol{v}])$.
   (f) Update $\boldsymbol{w}$:

$$w_k = \begin{cases} (1-\hat{\alpha})w_k + \hat{\gamma}\hat{\alpha} & , k = \hat{k} \\ (1-\hat{\alpha})w_k & , k \neq \hat{k}. \end{cases}$$

3. **Return** $\boldsymbol{w}$.

Figure 1. Coordinate-wise descent algorithm for generalized LASSO.

C^*. Kim and Kim (2004) proved the following theorem which gives an upper bound of $\Delta C(\boldsymbol{w}_m)$ in terms of the number of iterations of the CGD algorithm; see Kim and Kim (2004) for the proof.

Theorem 1. *Let $\boldsymbol{w}_m$ be the solution of generalized LASSO to be obtained by mth iteration of the CGD algorithm. Then,*

$$\Delta C(\boldsymbol{w}_m) \leq \frac{2M}{m}. \tag{3.2}$$

Remark. Note that the upper bound in Theorem 1 does not depend on the dimension of the inputs. Instead, it depends on the magnitudes of λ and $\boldsymbol{x}_i$s as well as y_is. To illustrate this more clearly, consider the standard LASSO problem, $\mathcal{L}(y, \boldsymbol{x}, \boldsymbol{\beta}) = (y - \boldsymbol{x}'\boldsymbol{\beta})^2$. In this case, $C(\boldsymbol{w}) = \sum_{i=1}^n (y_i - \lambda \boldsymbol{x}_i' \boldsymbol{w})^2$ and $\nabla(\boldsymbol{w}) = -2\lambda \sum_{i=1}^n \boldsymbol{x}_i (y_i - \lambda \boldsymbol{x}_i' \boldsymbol{w})$. It can be shown that $\nabla(\boldsymbol{w})$ satisfies (3.1) with $L = 4\lambda^2 \eta$ where η is the largest eigenvalue of $\sum_{i=1}^n \boldsymbol{x}_i \boldsymbol{x}_i'$. Since $\sup_{\boldsymbol{w}, \boldsymbol{v} \in \mathcal{S}} \|\boldsymbol{w} - \boldsymbol{v}\| \leq 2$, we can take $M_1 = 2L$. Let $\boldsymbol{w}^* = \operatorname{argmin}_{\boldsymbol{w} \in \mathcal{S}} C(\boldsymbol{w})$, Then

$$\begin{aligned} C(\boldsymbol{w}) - C(\boldsymbol{w}^*) &\leq -\langle \nabla(\boldsymbol{w}), \boldsymbol{w}^* - \boldsymbol{w} \rangle \\ &\leq \|\nabla(\boldsymbol{w})\| \|\boldsymbol{w}^* - \boldsymbol{w}\| \\ &\leq L \|\boldsymbol{w}\| \|\boldsymbol{w}^* - \boldsymbol{w}\| \\ &\leq 2L. \end{aligned}$$

We thus can take $M_2 = 2L$, so that $\Delta C(\boldsymbol{w}_m) \leq 4L/m$. Suppose that the column vectors of the design matrix have been standardized. Then, η is bounded by the sample size n.

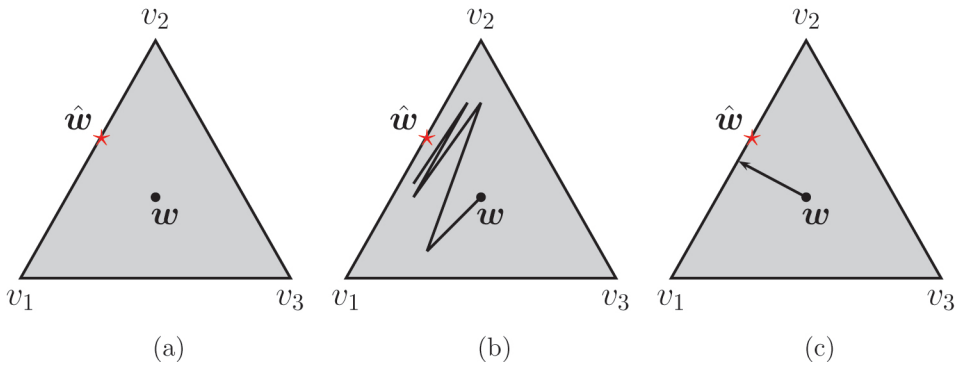


Figure 2. The first case of slow convergence for the CGD algorithm: (a) depicts the current solution (filled dot) and the optimal solution (star shape); (b) is a solution path of the CGD algorithm; (c) illustrates an ideal path of the algorithm to speed up.

Hence, L depends on n and λ but not on p. Note that, in each iteration, the CGD algorithm evaluates the gradient whose dimension is linearly proportional to p. This suggests that the overall computing time is linearly proportional to p, and so we can expect that the computational cost of the CGD algorithm is not high compared with other algorithms.

3.2 GRADIENT LASSO ALGORITHM

A problem with the CGD algorithm is that it converges slowly near the optimum. This is mainly because the CGD algorithm has only the addition step whereas Osborne's algorithm has the deletion step as well as the addition step. In this section, we explain reasons for the slow convergence of the CGD algorithm, and propose a method to improve the convergence speed by introducing the deletion step. The CGD algorithm with the deletion step is called the *gradient LASSO* algorithm.

Consider the problem of minimizing $C(\boldsymbol{w}) = \sum_{i=1}^{n} \mathcal{L}(y_i, \boldsymbol{x}_i, \boldsymbol{w})$ subject to $\|\boldsymbol{w}\|_1 \leq 1$. Let $\hat{\boldsymbol{w}}$ be the optimal solution, that is, $\hat{\boldsymbol{w}} = \text{argmin}_{\boldsymbol{w} \in \mathcal{S}} C(\boldsymbol{w})$ and $\boldsymbol{w}$ be the current solution provided by the CGD algorithm. Assuming that $\|\hat{\boldsymbol{w}}\|_1 = 1$, there exist two cases where the convergence of the CGD algorithm can be slow. The first is $\sigma(\boldsymbol{w}) - \sigma(\hat{\boldsymbol{w}}) \neq \emptyset$, and the second is $\sigma(\boldsymbol{w}) - \sigma(\hat{\boldsymbol{w}}) = \emptyset$ and $\|\boldsymbol{w}\|_1 < 1$. Here, $\sigma(\boldsymbol{w}) = \{j : w_j \neq 0, j = 1, \ldots, p\}$.

In the first case, where $\sigma(\boldsymbol{w}) - \sigma(\hat{\boldsymbol{w}}) \neq \emptyset$, the current solution includes some nonzero coefficients which should be zeros in $\hat{\boldsymbol{w}}$. In order to make nonzero coefficients in the current solution be zeros, the CGD algorithm adds other coefficients sequentially. In this situation, the convergence speed of the CGD algorithm can be very slow. For example, suppose $p = 3, \sigma(\hat{\boldsymbol{w}}) = \{1, 2\}$ and $\sigma(\boldsymbol{w}) = \{1, 2, 3\}$. Let $\boldsymbol{v}_1 = (1, 0, 0)', \boldsymbol{v}_2 = (0, 1, 0)'$ and $\boldsymbol{v}_3 = (0, 0, 1)'$. Then, the optimal solution is a convex combination of $\boldsymbol{v}_1$ and $\boldsymbol{v}_2$ while the current solution is a convex combination of $\boldsymbol{v}_1, \boldsymbol{v}_2$ and $\boldsymbol{v}_3$. Figure 2(a) depicts this situation where the filled dot is the current solution and the star shape indicates the optimal solution. The CGD algorithm must move from the filled dot to the star shape by adding $\boldsymbol{v}_1$ and $\boldsymbol{v}_2$ sequentially, but this requires a large number of iterations. A typical solution path of the CGD algorithm is shown in Figure 2(b). In this case, one way to speed up the

algorithm is to move the all coordinates simultaneously as in Figure 2(c). To do so, we should determine the direction and step size for the current solution to move to the next. Given vector $\boldsymbol{v} \in R^p$, let $\boldsymbol{v}_\sigma$ be the subvector of $\boldsymbol{v}$ defined by $\boldsymbol{v}_\sigma = (v_k, k \in \sigma(\boldsymbol{w}))$. That is,

$$\boldsymbol{v} = P^T \begin{pmatrix} \boldsymbol{v}_\sigma \\ \mathbf{0} \end{pmatrix},$$

where P is the permutation matrix that collects the nonzero elements of $\boldsymbol{w}$ in the first $|\sigma(\boldsymbol{w})|$ elements. For the direction, a natural choice is a negative gradient. Consider $\boldsymbol{w}_\sigma - \delta\nabla(\boldsymbol{w})_\sigma$ for some $\delta > 0$ as a next solution. A problem of using the negative gradient as the desirable direction is that the next solution may not be feasible for all $\delta > 0$. That is, $\|\boldsymbol{w}_\sigma - \delta\nabla(\boldsymbol{w})_\sigma\|_1 > 1$ for all $\delta > 0$. Moreover, it can be shown that this occurs when $\langle \nabla(\boldsymbol{w})_\sigma, \boldsymbol{\theta}(\boldsymbol{w})_\sigma \rangle < 0$ where $\boldsymbol{\theta}(\boldsymbol{w}) = (\text{sign}(w_1), \ldots, \text{sign}(w_p))'$. In this case, instead of using the gradient itself, we project the negative gradient $-\nabla(\boldsymbol{w})_\sigma$ onto the hyperplane $\mathcal{W}_\sigma = \{\boldsymbol{v} \in R^{|\sigma(\boldsymbol{w})|} : \sum_{k=1}^{|\sigma(\boldsymbol{w})|} v_k = 0\}$. Let $\boldsymbol{h}_\sigma$ be the projection vector of $-\nabla(\boldsymbol{w})_\sigma$ onto $\mathcal{W}_\sigma$ which is defined by $\boldsymbol{h}_\sigma = \text{argmin}_{\boldsymbol{v} \in \mathcal{W}_\sigma} \| - \nabla(\boldsymbol{w})_\sigma - \boldsymbol{v}\|^2$. It turns out that

$$\boldsymbol{h}_\sigma = -\nabla(\boldsymbol{w})_\sigma + \frac{\langle \boldsymbol{\theta}(\boldsymbol{w})_\sigma, \nabla(\boldsymbol{w})_\sigma \rangle}{|\sigma(\boldsymbol{w})|} \boldsymbol{\theta}(\boldsymbol{w})_\sigma.$$

In addition, $\|\boldsymbol{w} + \delta\boldsymbol{h}\|_1 = 1$ for $\delta \in [0, L]$ where

$$\boldsymbol{h} = P^T \begin{pmatrix} \boldsymbol{h}_\sigma \\ \mathbf{0} \end{pmatrix}, \tag{3.3}$$

and $L = \min_j\{-w_j/h_j : w_j h_j < 0, j \in \sigma(\boldsymbol{w})\}$. So, we update $\boldsymbol{w}$ by $\boldsymbol{w} + \hat{\delta}\boldsymbol{h}$ where $\hat{\delta} = \text{argmin}_{\delta \in [0,L]} C(\boldsymbol{w} + \delta\boldsymbol{h})$. When $\langle \nabla(\boldsymbol{w})_\sigma, \boldsymbol{\theta}(\boldsymbol{w})_\sigma \rangle \geq 0$, a constant $L > 0$ exists such that $\|\boldsymbol{w} + \delta\boldsymbol{h}\|_1 \leq 1$ for $\delta \in [0, L]$, where $\boldsymbol{h}_\sigma = -\nabla(\boldsymbol{w})_\sigma$ and $\boldsymbol{h}$ is defined by (3.3). However, it is difficult to determine the maximum value of L. Hence, instead of finding the maximum value of L, we let $L = \min_j\{-w_j/h_j : w_j h_j < 0, j \in \sigma(\boldsymbol{w})\}$, which is the smallest value when one of the coordinates of $\boldsymbol{w} + L\boldsymbol{h}$ in $\sigma(\boldsymbol{w})$ becomes zero. Thus, we can obtain $\hat{\delta} = \text{argmin}_{\delta \in [0,L]} C(\boldsymbol{w} + \delta\boldsymbol{h})$, and update $\boldsymbol{w}$ by $\boldsymbol{w} + \hat{\delta}\boldsymbol{h}$. Note that, when $\hat{\delta} = L$, one of the coordinates of $\boldsymbol{w} + \hat{\delta}\boldsymbol{h}$ in $\sigma(\boldsymbol{w})$ becomes zero. Hence, we can consider the process explained above as the deletion step. However, if $\hat{\delta} < L$, the deletion does not occur.

For the second case, where $\sigma(\boldsymbol{w}) - \sigma(\hat{\boldsymbol{w}}) = \emptyset$ and $\|\boldsymbol{w}\|_1 < 1$, the convergence speed is also slow just as in the first case. Figures 3(a), (b), and (c) illustrate this case for $p = 2$. Here $\boldsymbol{v}_0 = (0, 0)$, $\boldsymbol{v}_1 = (1, 0)$ and $\boldsymbol{v}_2 = (0, 1)$. Hence, we can improve the convergence speed as we did in the first case. However, the situation is simpler since a positive constant L always exists such that $\|\boldsymbol{w} + \delta\boldsymbol{h}\|_1 \leq 1$ for $\delta \in [0, L]$ where $\boldsymbol{h}$ is defined by (3.3) with $\boldsymbol{h}_\sigma = -\nabla(\boldsymbol{w})_\sigma$. We can choose $L = \min_j\{-w_j/h_j : w_j h_j < 0, j \in \sigma(\boldsymbol{w})\}$ similarly as in the first case. Finally, we update $\boldsymbol{w}$ by $\boldsymbol{w} + \hat{\delta}\boldsymbol{h}$ where $\hat{\delta} = \text{argmin}_{\delta \in [0,L]} C(\boldsymbol{w} + \delta\boldsymbol{h})$.

By combining the CGD algorithm and the proposed deletion step, we propose the gradient LASSO algorithm given in Figure 4. In the gradient LASSO algorithm, we need to solve two one-dimensional convex optimization problems, one for $\hat{\alpha}$ in the CGD algorithm

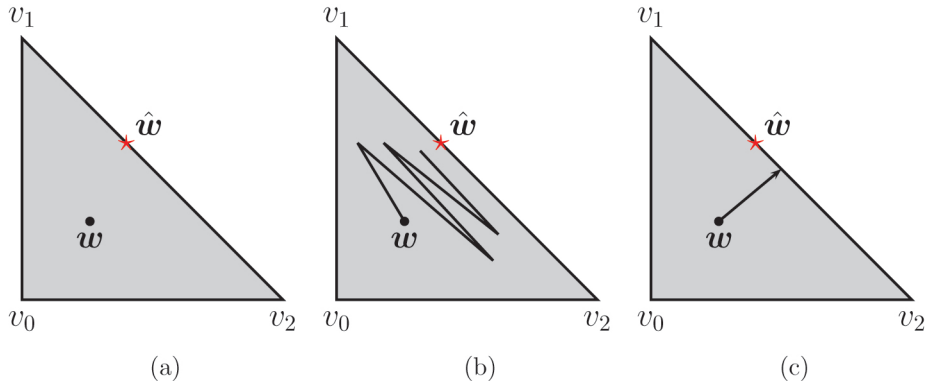


Figure 3. The second case of slow convergence for the CGD algorithm: (a) depicts the current solution (filled dot) and the optimal solution (star shape); (b) is a solution path of the CGD algorithm; (c) illustrates an ideal path of the algorithm to speed up.

1. **Initialize**: $\boldsymbol{w} = \mathbf{0}$ and $m = 0$.
2. **Do until converge**
   (a) **Addition step (the CGD algorithm)**:
      i. Update $\boldsymbol{w}$ using the CGD algorithm.
   (b) **Deletion step**:
      i. Let $\nabla(\boldsymbol{w})_\sigma = (\nabla(\boldsymbol{w})_k, k \in \sigma(\boldsymbol{w}))$.
      ii. If $\langle \nabla(\boldsymbol{w})_\sigma, \boldsymbol{\theta}(\boldsymbol{w})_\sigma \rangle < 0$ and $\langle \boldsymbol{\theta}(\boldsymbol{w}), \boldsymbol{w} \rangle = 1$, set
$$\boldsymbol{h}_\sigma = -\nabla(\boldsymbol{w})_\sigma + \frac{\langle \boldsymbol{\theta}(\boldsymbol{w})_\sigma, \nabla(\boldsymbol{w})_\sigma \rangle}{|\sigma(\boldsymbol{w})|} \boldsymbol{\theta}(\boldsymbol{w})_\sigma.$$
      iii. Else, set
$$\boldsymbol{h}_\sigma = -\nabla(\boldsymbol{w})_\sigma.$$
      iv. Let $\boldsymbol{h}$ be the vector defined by (3.3) with $\boldsymbol{h}_\sigma$ and the corresponding permutation matrix P.
      v. Compute $\hat{\delta} = \arg\min_{0<\delta<L} C(\boldsymbol{w} + \delta \boldsymbol{h})$, where $L = \min\{-w_j / h_j : w_j h_j < 0, j \in \sigma(\boldsymbol{w})\}$.
      vi. Update $\boldsymbol{w} = \boldsymbol{w} + \hat{\delta} \boldsymbol{h}$.
   (c) $m = m + 1$.
3. **Return** $\boldsymbol{w}$.

Figure 4. Gradient LASSO algorithm

and the other for $\hat{\delta}$ in the deletion step. We used a line search based method, which does not greatly increase the overall computing time and never fails to find the solution.

Note that the cost function always decreases as the deletion step goes on, and hence the gradient LASSO algorithm always converges to the global optimum. Furthermore, the results in Section 4.1 suggest that the deletion step can reduce the overall computing time considerably.

4. ILLUSTRATIONS

4.1 COMPARISON OF THE CGD AND GRADIENT LASSO ALGORITHMS

Figure 5 shows the trajectories of the training errors (the empirical risk divided by the sample size) and the number of zero coefficients over the number of iterations for the gradient LASSO and CGD algorithms, respectively. For this result, we generated a dataset of size 20 from the multiple regression model with $p = 50$ and the constraint $\sum_{i=1}^{50} |\beta_i| \leq 3$. For input variables and errors, we used independent standard normal distributions. We applied the algorithms to two different initial models: the null model with all coefficients being set to 0; the full model with all coefficients being set to 3/50. In Figure 5, we can see that the gradient LASSO algorithm converges faster than the CGD algorithm, regardless of the choice of initial models. In particular, with the null initial model, the gradient LASSO algorithm reaches to the minimum just after 20 iterations while the CGD algorithm does not so even after 200 iterations.

The number of zero coefficients strongly depends on the initial model. With the null initial model, the gradient LASSO algorithm gives slightly more sparse solution than the CGD algorithm does, but their difference is negligible. In contrast, with the full initial model, the gradient LASSO algorithm successfully deletes insignificant coefficients, while the CGD algorithm fails to do so. This observation implies that the deletion step in the gradient LASSO algorithm is necessary for obtaining a sparse solution. Moreover, note that the numbers of zero coefficients can be quite different from each other even if the training errors are very close. This suggests that, in order to develop a sparse learning algorithm, approximation of the training error is insufficient for variable selection.

4.2 COMPARISON OF THE GRADIENT LASSO AND `lasso2` ALGORITHM

In this section, we compare the gradient LASSO algorithm with the `lasso2` algorithm of Lokhorst et al. (2006) in terms of accuracy, sparseness, computing time and stability. For accuracy, we computed the empirical risk of the final solution; for sparseness, we counted the number of zero coefficients; the CPU time was measured using `system.time()` in the R system; the stability was measured by counting the number of simulations where the algorithm converges. For the gradient LASSO algorithm, we implemented the R package which is available at *http://idea.snu.ac.kr/Research/glassojskim/glasso.htm*. In our experiments, the gradient LASSO algorithm stopped when the relative change of the L_1 norm of the coefficients is less than 10^{-8}.

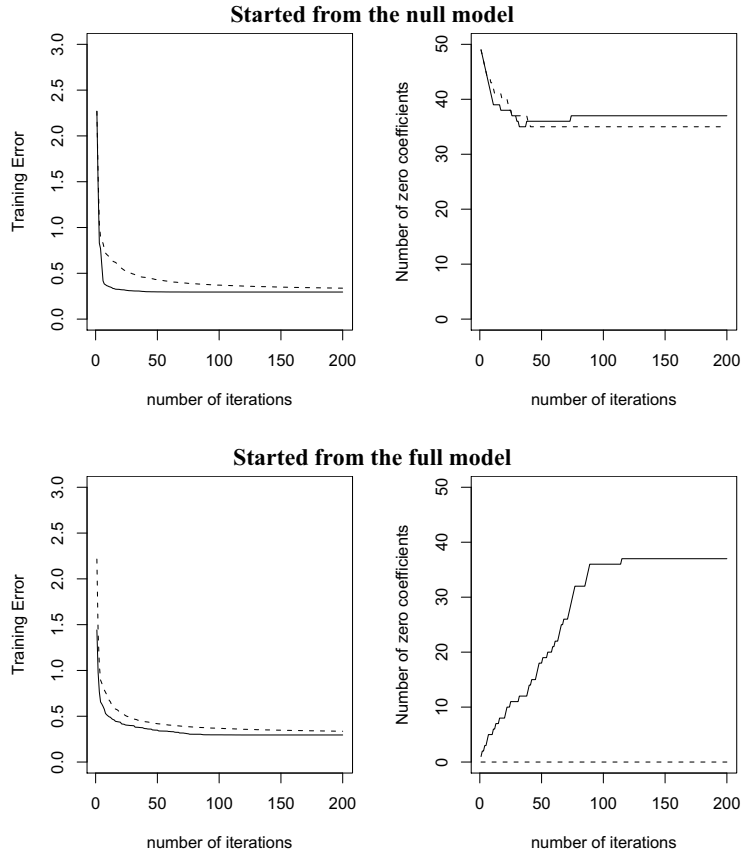


Figure 5. Trajectories of the training errors and numbers of zero coefficients from the gradient LASSO (solid) and CGD (dashed) algorithm.

We considered the following multiple logistic regression model:

$$f(\boldsymbol{z}) = \log\left(\frac{P(y=1|\boldsymbol{z})}{1-P(y=1|\boldsymbol{z})}\right) = \beta_0 + \beta_1 z_1 + \cdots + \beta_p z_p,$$

where z_i's are independent standard normal random variables. Experiments were performed under the different simulation settings: numbers of inputs ($p = 10, 20, 50$, and 100) and constraints ($\lambda = 1, 2, 5$, and 10). We set $\beta_0 = 1.0, \beta_1 = 1.0, \beta_2 = 0.5, \beta_3 = -0.5$ and the other remaining coefficients to zeros. Under each simulation setting, we repeated the experiment 100 times. Table 1 summaries the results of experiments where '# of NA' is the number of failures of the algorithm in 100 runs, "Comput.time" indicates the average computing time, "Training error" means the average of the empirical risks divided by the sample size, and "# of zero coeff" is the average number of zero coefficients.

The training errors of the gradient LASSO algorithm seem to be slightly larger than those of the `lasso2` algorithm, in particular when λ is large. This is because the gradient LASSO algorithm converges the the optimal solution as the number of iterations goes to infinity, but in practice we terminate the algorithm after finitely many iterations. Nonethe-

Table 1. Comparison of the gradient LASSO and `lasso2` algorithms on the multiple logistic regression model: The numbers in parentheses are the standard errors. The results marked by (*) are summarized only for the case when the `lasso2` algorithm does not fail.

p	λ	Method	# of NA	Comput. time	Training error	# of zero coeff.
10	1	QP	0	0.02 (0.00)	0.4557 (0.0069)	6.72 (0.12)
		gradient	0	0.08 (0.00)	0.4557 (0.0069)	6.72 (0.12)
	2	QP	0	0.02 (0.00)	0.3698 (0.0079)	5.25 (0.15)
		gradient	0	0.09 (0.00)	0.3698 (0.0079)	5.26 (0.15)
	5	QP	0	0.02 (0.00)	0.2432 (0.0099)	3.15 (0.15)
		gradient	0	0.12 (0.00)	0.2432 (0.0099)	3.14 (0.15)
	10	QP	0	0.02 (0.00)	0.1599 (0.0113)	2.08 (0.14)
		gradient	0	0.19 (0.01)	0.1599 (0.0113)	2.04 (0.14)
20	1	QP	0	0.02 (0.00)	0.4327 (0.0063)	16.34 (0.14)
		gradient	0	0.10 (0.00)	0.4327 (0.0063)	16.34 (0.14)
	2	QP	0	0.02 (0.00)	0.3289 (0.0064)	14.33 (0.15)
		gradient	0	0.11 (0.00)	0.3289 (0.0064)	14.33 (0.15)
	5	QP	0	0.03 (0.00)	0.1621 (0.0058)	11.53 (0.19)
		gradient	0	0.19 (0.01)	0.1621 (0.0058)	11.55 (0.18)
	10	QP	0	0.03 (0.00)	0.0568 (0.0038)	10.27 (0.17)
		gradient	0	0.28 (0.01)	0.0568 (0.0038)	10.27 (0.17)
50	1	QP	0	0.04 (0.00)	0.4240 (0.0057)	45.38 (0.14)
		gradient	0	0.13 (0.00)	0.4240 (0.0057)	45.38 (0.14)
	2	QP	0	0.04 (0.00)	0.3050 (0.0054)	42.90 (0.17)
		gradient	0	0.16 (0.01)	0.3050 (0.0054)	42.90 (0.17)
	5	QP	0	0.04 (0.00)	0.1173 (0.0038)	39.49 (0.19)
		gradient	0	0.28 (0.01)	0.1173 (0.0038)	39.47 (0.19)
	10	QP(*)	29	0.04 (0.00)	0.0242 (0.0018)	37.63 (0.22)
		gradient(*)	0	0.48 (0.03)	0.0242 (0.0018)	37.49 (0.23)
100	1	QP	0	0.06 (0.00)	0.4019 (0.0066)	95.03 (0.16)
		gradient	0	0.18 (0.00)	0.4019 (0.0066)	95.02 (0.16)
	2	QP	0	0.06 (0.00)	0.2715 (0.0057)	92.39 (0.20)
		gradient	0	0.22 (0.01)	0.2715 (0.0057)	92.37 (0.20)
	5	QP(*)	6	0.06 (0.00)	0.0853 (0.0029)	88.77 (0.20)
		gradient(*)	0	0.36 (0.01)	0.0853 (0.0029)	88.72 (0.21)
	10	QP(*)	63	0.07 (0.00)	0.0107 (0.0011)	87.35 (0.32)
		gradient(*)	0	0.48 (0.02)	0.0108 (0.0011)	86.92 (0.33)

less, the differences between the training errors seem to be nearly negligible.

For sparseness, the gradient LASSO algorithm gives slightly less sparse solutions than the `lasso2` algorithm does. When p and λ are large, the gradient LASSO algorithm sometimes produces the number of nonzero coefficients exceeding the sample size. This is because multiple solutions can exist in such cases. However, note that, when λ and p are large, the `lasso2` algorithm fails frequently while the gradient LASSO algorithm always gives a solution.

The computing times of the gradient LASSO are larger than those of the `lasso2` algorithm. This is not surprising since, as we mentioned earlier, the gradient LASSO algorithm converges the the optimal solution as the number of iterations goes to infinity. However, we believe that the gradient LASSO algorithm is fast enough for practical purposes. We

Table 2. Training and test errors for various constrained parameters for the Leukemia data set: The numbers in the parenthesis are CPU times in seconds.

	Training error		Test error
λ	gradient LASSO	`lasso2`	gradient LASSO
1	0.316852395 (18.88)	0.31685205 (19.16)	0.4686584
2	0.160236853 (19.65)	0.16023674 (20.37)	0.3127428
3	0.079347152 (20.35)	0.07934697 (19.25)	0.2507894
4	0.038937536 (19.05)	0.03893365 (20.59)	0.2151390
5	0.018990136 (21.53)	NA	0.1811580
6	0.009252764 (21.09)	NA	0.1662746
7	0.004512818 (20.54)	NA	0.1531072
8	0.002203785 (21.18)	NA	0.1506219
9	0.001077669 (21.54)	NA	0.1615514

again emphasize that the objective of the gradient LASSO algorithm is not to reduce its computational cost, but to achieve stability in the sense that it never fails to converge when p is larger than n.

One may argue that the algorithm fails to converge only when λ is unnecessarily large, and so the failure of convergence would not hamper the prediction accuracy as long as λ has been selected wisely. We will show that this argument is not always true by analyzing the Leukemia dataset, a popular microarray dataset consisting of expression levels for 7,129 genes. Each sample has one of two classes: acute lymphoblastic leukemia (ALL) or acute myeloid leukemia (AML). The dataset was analyzed by Golub et al. (1999), where it was divided into a training set (38 samples) and a test set (34 samples). The dataset is available on the website (*http://www.broad.mit.edu/cgi-bin/cancer/datasets.cgi*). Table 2 summarizes the training and test errors for various values of the constrained parameter λ. The minimum test error is achieved when $\lambda = 8$, but the `lasso2` algorithm fails to converge when $\lambda > 4$. In contrast, the gradient LASSO algorithm does not fail at all. This analysis implies that the `lasso2` algorithm can fail too early in terms of λ, which results in inferior prediction accuracy. Hence, developing an algorithm that does not fail for high-dimensional data is important not only for stability, but also for accuracy. In addition, note that for small values of λ, the gradient LASSO algorithm converges even faster than the `lasso2` algorithm.

4.3 ANALYSIS OF GENE EXPRESSION DATA USING THE SPARSE MULTICLASS LOGISTIC REGRESSION

We analyzed the NCI60 dataset using sparse multiclass logistic regression. The NCI60 data set consists of gene expression profiles of 60 National Cancer Institute (NCI) cell lines. These 60 human tumor cell lines are derived from patients with leukemia and melanoma as well as lung, colon, central nervous system, ovarian, renal, breast, and prostate cancers. The data set comprises of gene-expression levels of $p = 7,129$ genes for $n = 60$ human tumor cell lines that can be divided into 8 classes: 8 breast, 6 CNS, 7 colon, 6 leukemia, 8 melanoma, 9 non-small-cell lung carcinoma, 6 ovarian and 8 renal tu-

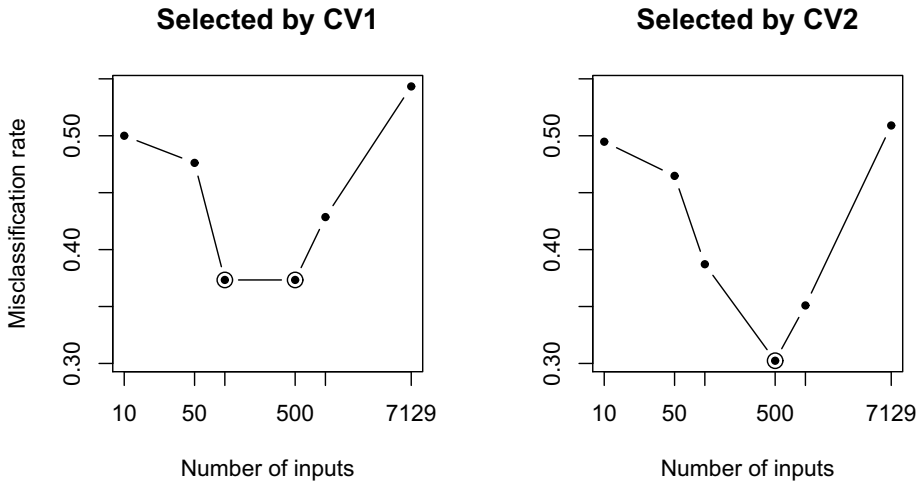


Figure 6. Misclassification error rates estimated by CV_1(left) or CV_2(right) vs. the number of prescreened inputs: The circles are marked at their minima.

mors. A more detailed description of the data is found at Staunton et al. (2001). One can download the data set at the website of Genomics & Bioinformatics Group (*http://discover.nci.nih.gov/datasetsPnas2001.jsp*).

The main purpose of this analysis is to investigate the effect of gene prescreening on the prediction accuracy. One of the standard approaches for analyzing gene expression data starts with picking out relevant genes in order to reduce computational costs as well as to improve prediction accuracy. For this, simple prescreening measures have been used. See for example, Golub et al. (1999) and Dudoit et al. (2002). Since multiclass problems are our current concern, we used the F-ratio as a prescreening measure. Following Dudoit et al. (2002), the F-ratio for a gene l is defined by the ratio of between-class sum of squares to within-class sum of squares:

$$\frac{\mathrm{BSS}(l)}{\mathrm{WSS}(l)} = \frac{\sum_{i=1}^{n}\sum_{j=1}^{J} I(y_i = j)(\bar{x}_{.l}^{(j)} - \bar{x}_{.l})^2}{\sum_{i=1}^{n}\sum_{j=1}^{J} I(y_i = j)(x_{il} - \bar{x}_{.l}^{(j)})^2},$$

where $\bar{x}_{.l}^{(j)}$ indicates the average expression level of a gene l in samples belonging to j-class, and $\bar{x}_{.l}$ is the overall mean expression level of a gene l in the dataset.

For estimating the prediction error, we partitioned the dataset randomly into two subsets; we used 70% of the dataset as a training set and 30% of the dataset as a test set. We repeated this procedure 100 times and obtained the average misclassification error. In each repetition, we selected the optimal constrained parameter λ using the five-fold cross-validation. For the cross-validation, we used the two measurements, the 0-1 loss (CV_1) and the negative log-likelihood loss (CV_2).

Figure 6 shows the misclassification error rates. For this results, we first ranked genes according to the F-ratios, then selected the top ranked genes of sizes—10, 50, 100, 500, 1000, and 7129. Using each group of selected genes as prescreening inputs, we ran the

gradient LASSO algorithm and computed the misclassification errors. The minimum error rate in Figure 6 is achieved for 500 genes with CV_2. This result suggests that the purpose of prescreening is not to select relevant genes but to eliminate irrelevant ones. This conclusion appears to contradict that of Dudoit et al. (2002) in which finding small numbers of relevant genes by prescreening affected the prediction accuracy significantly in some cases. A reason for this difference could be that we used a sparse learning method while Dudoit et al. (2002) did not. For nonsparse learning methods, the classifier depends on all genes used as inputs and so prescreening would be important. However, sparse learning methods automatically select genes while they construct classifiers, and so prescreening would not be necessary. Moreover, in some cases, the prescreening can drop some informative genes in early stages, which may result in a suboptimal model. In our data analysis, the results with less than 500 prescreening genes were suboptimal. From this perspective, efficient computational algorithms dealing with high dimensional inputs should be necessary for sparse learning methods, and the gradient LASSO algorithm can be one of such algorithms.

5. CONCLUSIONS

In this article, we proposed a new computational algorithm for generalized LASSO called the gradient LASSO algorithm. The proposed algorithm is computationally more simple and stable than the standard QP-based algorithms. In particular, it never fails to converge even when $p > n$. We also showed theoretically as well as empirically that the proposed algorithm converges quickly and gives reliable results.

Along with finding the solution of generalized LASSO for a fixed value of λ, we are interested in finding the set of solutions for various values of λ. Let $\hat{\boldsymbol{\beta}}(\lambda)$ be the solution of the generalized LASSO problem with $\|\boldsymbol{\beta}\|_1 \leq \lambda$. Many researchers—including Efron et al. (2004), Rosset (2005), Rosset et al. (2004), and Zhao and Yu (2004)—have studied methods of finding $\hat{\boldsymbol{\beta}}(\lambda)$ for all λ, which is called the *regularized solution path*. Unfortunately, the gradient LASSO algorithm does not give the regularized solution path automatically. A way of finding an approximated regularized solution path is to evaluate $\hat{\boldsymbol{\beta}}(\lambda)$ at $\lambda = \epsilon, 2\epsilon, 3\epsilon, \ldots$ for sufficiently small values of $\epsilon > 0$. To improve the computational speed, we can use $\hat{\boldsymbol{\beta}}(\lambda)$ as the initial solution when we are to find $\hat{\boldsymbol{\beta}}(\lambda + \epsilon)$. Note that even though the exact regularized solution path is obtained by Osborne et al. (2000a) and Efron et al. (2004) for the squared error loss, none of the algorithms find the exact regularized solution paths for general loss functions.

Examples of other sparse learning methods include fused LASSO (Tibshirani et al. 2005), grouped LASSO (Yuan and Lin 2006), blockwise sparse regression (Kim et al. 2006), SCAD (Fan and Li 2001), and elastic net (Zou and Hastie 2005). As shown in Subsection 4.1, solutions which minimize the empirical risk crudely in sparse learning methods could be poor in variable selection. Hence, we will pursue Hence, further pursuing the development of efficient and globally convergent computational algorithms for these methods, particularly for large dimensional data, is warranted.

ACKNOWLEDGMENTS

We thank the Associate Editor and the referees whose helpful comments led to significant improvements in this paper. This research was supported in part by the Korean Foundation grant KRF-2005-070-C00021 and KRF-2005-214-C00187 funded by the Korean Government (MOEHRD).

[Received May 2007. Revised January 2008.]

REFERENCES

Bakin, S. (1999), "Adaptive Regression and Model Selection in Data Mining Problem," PhD thesis, Australian National University, Australia.

Barron, A. R. (1993), "A Universal Approximation Bounds for Superpositions of a Sigmoidal Function," *IEEE Transactions on Information Theory*, 39, 930–945.

Chen, S. S., Donoho, D. L., and Saunders, M. A. (1999), "Atomic Decomposition by Basis Pursuit," *SIAM Journal on Scientific Computing*, 20, 33–61.

Dudoit, S., Fridlyand, J., and Speed, T. (2002), "Comparison of Discrimination Methods for the Classification of Tumors using Gene Expression Data," *Journal of the American Statistical Association*, 97, 77–87.

Efron, B., Hastie, T., Johnstone, I., and Tibshirani, R. (2004), "Least Angle Regression," *The Annals of Statistics*, 32, 407–499.

Fan, J., and Li, R. (2001), "Variable Selection via Noncocave Penalized Likelihood and its Oracle Properties," *Journal of the American Statistical Association*, 96, 1348–1360.

Golub, T., Slonim, D., Tamayo, P., Huard, C., Gaasenbeek, M., Mesirov, J., Coller, H., Loh, M., Downing, J., Caligiuri, M., Bloomfield, C., and Lander, E. (1999), "Molecular Classification of Cancer: Class Discovery and Class Prediction by Gene Expression Monitoring," *Science*, 286, 531–537.

Grandvalet, Y., and Canu, S. (1999), "Outcomes of the Equivalence of Adaptive Ridge with Least Absolute Shrinkage," in *Advances in Neural Information Processing Systems*, eds. M. Kearns, S. Solla, and D. Cohn, vol. 11, Cambridge, MA: MIT Press, pp. 445–451.

Gunn, S. R., and Kandola, J. S. (2002), "Structural Modelling with Sparse Kernels," *Machine Learning*, 48, 115–136.

Jones, L. K. (1992), "A Simple Lemma on Greedy Approximation in Hilbert Space and Convergence Rates for Projection Pursuit Regression and Neural Network Training," *The Annals of Statistics*, 20, 608–613.

Kim, Y., and Kim, J. (2004), "Gardient Lasso for Feature Selection," in *Proceedings of the 21th International Conference on Machine Learning*, New York: Morgan Kaufmann, pp. 473–480.

Kim, Y., Kim, J., and Kim, Y. (2006), "Blockwise Sparse Regression," *Statistica Sinica*, 16, 375–390.

Krishnapuram, B., Carlin, L., Figueiredo, M., and Hartemink, A. (2004), "Learning Sparse Classifier: Multi-class Formulation, Fast Algorithms and Generalization Bounds," *IEEE Tranactions on Pattern Analysis and Machine Intelligence*, 27, 957–968.

Lokhorst, J., Turlach, B. A., and Venables, W. N. (2006), "Lasso2*: An S-plus Library to Solve Regression Problems While Imposing an L_1 Constraint on the Parameters," *Documentation in R-lasso2 library*.

Ma, S., and Huang, J. (2005), "Lasso Method for Additive Risk Models with High-Dimensional Covariates," Technical Report, Department of Statistics and Actuarial Science, The University of Iowa.

Osborne, M. R., Presnell, B., and Turlach, B. A. (2000a), "A New Approach to Variable Selection in Least Squares Problems," *IMA Journal of Numerical Analysis*, 20, 389–404.

——— (2000b), "On the LASSO and its Dual," *Journal of the Computational and Graphical Statistics*, 9, 319–337.

Park, M. Y., and Hastie, T. (2007), "L_1-Regularization Path Algorithm for Generalized Linear Models," *Journal of the Royal Statistical Society*, Series B, 69, 659–677.

Perkins, S., Lacker, K., and Theiler, J. (2003), "Grafting: Fast, Incremetal Feature Selection by Gradient Descent in Function Space," *Journal of Machine Learning Research*, 3, 1333–1356.

Rosset, S. (2005), "Following Curved Regularized Optimization Solution Paths," in *Advances in Neural Information Processing Systems 17*, eds. L. K. Saul, Y. Weiss, and L. Bottou, Cambridge, MA: MIT Press, pp. 1153–1160.

Rosset, S., and Zhu, J. (2007), "Piecewise Linear Regularized Solution Path," *The Annals of Statistics*, 35, 1012–1030.

Rosset, S., Zhu, J., and Hastie, T. (2004), "Boosting as a Regularized Path to a Maximum Margin Classifier," *Journal of Machine Learning Research*, 5, 941–973.

Roth, V. (2004), "The Generalized Lasso," *IEEE Transactions on Neural Networks*, 15, 16–28.

Staunton, J. E., Slonim, D. K., Coller, H. A., Tamayo, P., Angelo, M. J., Park, J., Scherf, U., Lee, J. K., Reinhold, W. O., Weinstein, J. N., Mesirov, J. P., Lander, E. S., and Golub, T. R. (2001), "Chemosensitivity Prediction by Transcriptional Profiling," in *Proceedings of the National Academy of Science*, vol. 98, pp. 10787–10792.

Tibshirani, R. (1996), "Regression Shrinkage and Selection via the Lasso," *Journal of the Royal Statistical Society*, Series B, 58, 267–288.

Tibshirani, R., Saunders, M., Rosset, S., Zhu, J., and Knight, K. (2005), "Sparsity and Smoothness via the Fused Lasso," *Journal of the Royal Statistical Society*, Series B, 67, 91–108.

Yuan, M., and Lin, Y. (2006), "Model Selection and Estimation in Regression with Grouped Variables," *Journal of the Royal Statistical Society*, Series B, 68, 49–67.

Zhang, H. H., Wahba, G., Lin, Y., Voelker, M., Ferris, M., Klein, R., and Klein, B. (2004), "Variable Selection and Model Building via Likelihood Basis Pursuit," *Journal of the American Statistical Association*, 99, 659–672.

Zhang, T. (2003), "Sequential Greedy Approximation for Certain Convex Optimization Problems," *IEEE Transactions on Information Theory*, 49, 682–691.

Zhao, P., and Yu, B. (2004), "Boosted Lasso," Technical Report 678, Department of Statistics, UC Berkeley.

Zou, H., and Hastie, T. (2005), "Regularization and Variable Selection via Elastic Net," *Journal of the Royal Statistical Society*, Series B, 67, 301–320.